Experiment 10 — SonarQube Static Code Analysis Lab

Objective

To understand and perform static code analysis using SonarQube by:
- Running a SonarQube server using Docker
- Scanning a Java project using Sonar Scanner (via Maven plugin)
- Viewing detected bugs, vulnerabilities, and code smells in the dashboard

---

Theory

Problem Statement

Code bugs and security vulnerabilities are often discovered too late in the development cycle. Manual reviews are slow and hard to scale.

Static code analysis tools like **SonarQube** automatically analyze source code without executing it.

What is SonarQube?

SonarQube is an open-source platform used for **continuous inspection of code quality**.

It detects:

- Bugs
- Security vulnerabilities
- Code smells
- Code duplication
- Technical debt
- Test coverage

SonarQube Architecture

SonarQube has two main components.

SonarQube Server

Responsible for:

- storing results
- applying analysis rules
- enforcing quality gates
- providing a dashboard

Runs at:

```
http://localhost:9000
```

Sonar Scanner

Scanner:

- reads source code
- gathers project metadata
- sends results to the server

The **server performs the real analysis**, not the scanner.

Architecture Flow

```
Source Code
     |
     ▼
Sonar Scanner
     |
     ▼
SonarQube Server
     |
     ▼
PostgreSQL Database
     |
     ▼
Web Dashboard
```

Step 1 — Start SonarQube Server

Create `docker-compose.yml`

```
yuvraj_singh@LUCIF3R: ~/

GNU nano 7.2                                                      docker-compose.yml *
version: '3.8'

services:

  # ── Database ──────────────────────────────────────────
  sonar-db:
    image: postgres:13
    container_name: sonar-db
    environment:
      POSTGRES_USER: sonar          # DB username
      POSTGRES_PASSWORD: sonar      # DB password
      POSTGRES_DB: sonarqube        # DB name
      POSTGRES_HOST_AUTH_METHOD: trust  # allow connections without password (for simplicity in this lab)
    volumes:
      - sonar-db-data:/var/lib/postgresql/data    # persist data across restarts
    networks:
      - sonarqube-lab

  # ── SonarQube Server ──────────────────────────────────
  sonarqube:
    image: sonarqube:lts-community
    container_name: sonarqube
    ports:
      - "9000:9000"                 # access dashboard at http://localhost:9000
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
    volumes:
      - sonar-data:/opt/sonarqube/data
      - sonar-extensions:/opt/sonarqube/extensions
    depends_on:
      - sonar-db                    # wait for DB to start first
    networks:
      - sonarqube-lab

# Named volumes (Docker manages storage location)
volumes:
  sonar-db-data:
  sonar-data:
  sonar-extensions:

# Isolated network so containers can talk to each other by name
networks:
  sonarqube-lab:
    driver: bridge
```

Start containers:

```
yuvraj chaula@LUCIF3R /Lab9s docker compose up ~d
WARN[0000] Ihome/yuvraj_ singh/Lab9/docker compose ymt: the attribute version is obsolete it will be ignored, please remove it to avoid pote
ntial confusion
[+] up 34/34
 ✔Image sonarqube:lts-community Pulled                                                                                         131.3s
 ✔Image postgres:13             Pulled                                                                                         111.7s
 ✔Network lab9_sonarqube-lab    Created                                                                                          0.0s
 ✔Volume lab9_sonar-data        Created                                                                                          0.0s
 ✔Volume lab9_sonar-extensions  Created                                                                                          0.0s
 ✔Volume lab9_sonar-db-data     Created                                                                                          0.0s
 ✔Container sonar-db            Started                                                                                          0.5s
 ✔Container sonarqube           Started                                                                                          0.4s
```

Check logs:

```
yuvraj singh@LUCIF3R /Lab9: docker compose logs ~f sonarqube
MARN[OOOo] /home_ yuvraj singh/Lab9/docker compose .yml the attribute version is obsolete it will be ignored, please remove it to avoid pote
ntial confusion
sonarqube  | 2026.04.29 17:28:16 INFO  app[][o.s.a.AppFileSystem] Cleaning or creating temp directory /opt/sonarqube/temp
sonarqube  | 2026.04.29 17:28:16 INFO  app[][o.s.a.es.EsSettings] Elasticsearch listening on [HTTP: 127.0.0.1:9001, TCP: 127.0.0.1:34159]
sonarqube  | 2026.04.29 17:28:16 INFO  app[][o.s.a.ProcessLauncherImpl] Launch process[ELASTICSEARCH] from [/opt/sonarqube/elasticsearch]: /opt/
sonarqube/elasticsearch/bin/elasticsearch
sonarqube  | 2026.04.29 17:28:16 INFO  app[][o.s.a.SchedulerImpl] Waiting for Elasticsearch to be up and running
sonarqube  | 2026.04.29 17:28:17 INFO  es[][o.e.n.Node] version[7.17.15], pid[29], build[default/tar/0b8ecfb4378335f4689c4223d1f1115f16bef3ba/20
23-11-10T22:03:46.987399016Z], OS[Linux/6.6.87.2-microsoft-standard-WSL2/amd64], JVM[Eclipse Adoptium/OpenJDK 64-Bit Server VM/17.0.16/17.0.16+8
]
sonarqube  | 2026.04.29 17:28:17 INFO  es[][o.e.n.Node] JVM home [/opt/java/openjdk]
sonarqube  | 2026.04.29 17:28:17 INFO  es[][o.e.n.Node] JVM arguments [-XX:+UseG1GC, -Djava.io.tmpdir=/opt/sonarqube/temp, -XX:ErrorFile=/opt/so
narqube/logs/es_hs_err_pid%p.log, -Des.networkaddress.cache.ttl=60, -Des.networkaddress.cache.negative.ttl=10, -XX:+AlwaysPreTouch, -Xss1m, -Dja
va.awt.headless=true, -Dfile.encoding=UTF-8, -Djna.nosys=true, -Djna.tmpdir=/opt/sonarqube/temp, -XX:-OmitStackTraceInFastThrow, -Dio.netty.noUn
safe=true, -Dio.netty.noKeySetOptimization=true, -Dio.netty.recycler.maxCapacityPerThread=0, -Dio.netty.allocator.numDirectArenas=0, -Dlog4j.shu
tdownHookEnabled=false, -Dlog4j2.disable.jmx=true, -Dlog4j2.formatMsgNoLookups=true, -Djava.locale.providers=COMPAT, -Dcom.redhat.fips=false, -D
es.enforce.bootstrap.checks=true, -Xmx512m, -Xms512m, -XX:MaxDirectMemorySize=256m, -XX:+HeapDumpOnOutOfMemoryError, -Des.path.home=/opt/sonarqu
be/elasticsearch, -Des.path.conf=/opt/sonarqube/temp/conf/es, -Des.distribution.flavor=default, -Des.distribution.type=tar, -Des.bundled_jdk=fal
se]
sonarqube  | 2026.04.29 17:28:17 INFO  es[][o.e.p.PluginsService] loaded module [analysis-common]
sonarqube  | 2026.04.29 17:28:17 INFO  es[][o.e.p.PluginsService] loaded module [lang-painless]
```

Open dashboard:

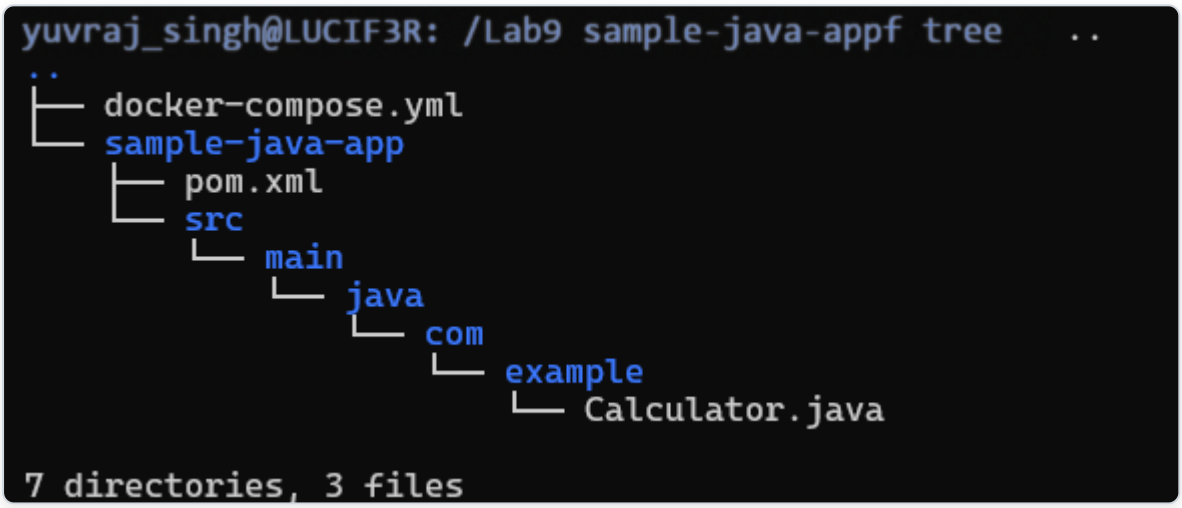
```
http://localhost:9000
```

Login:

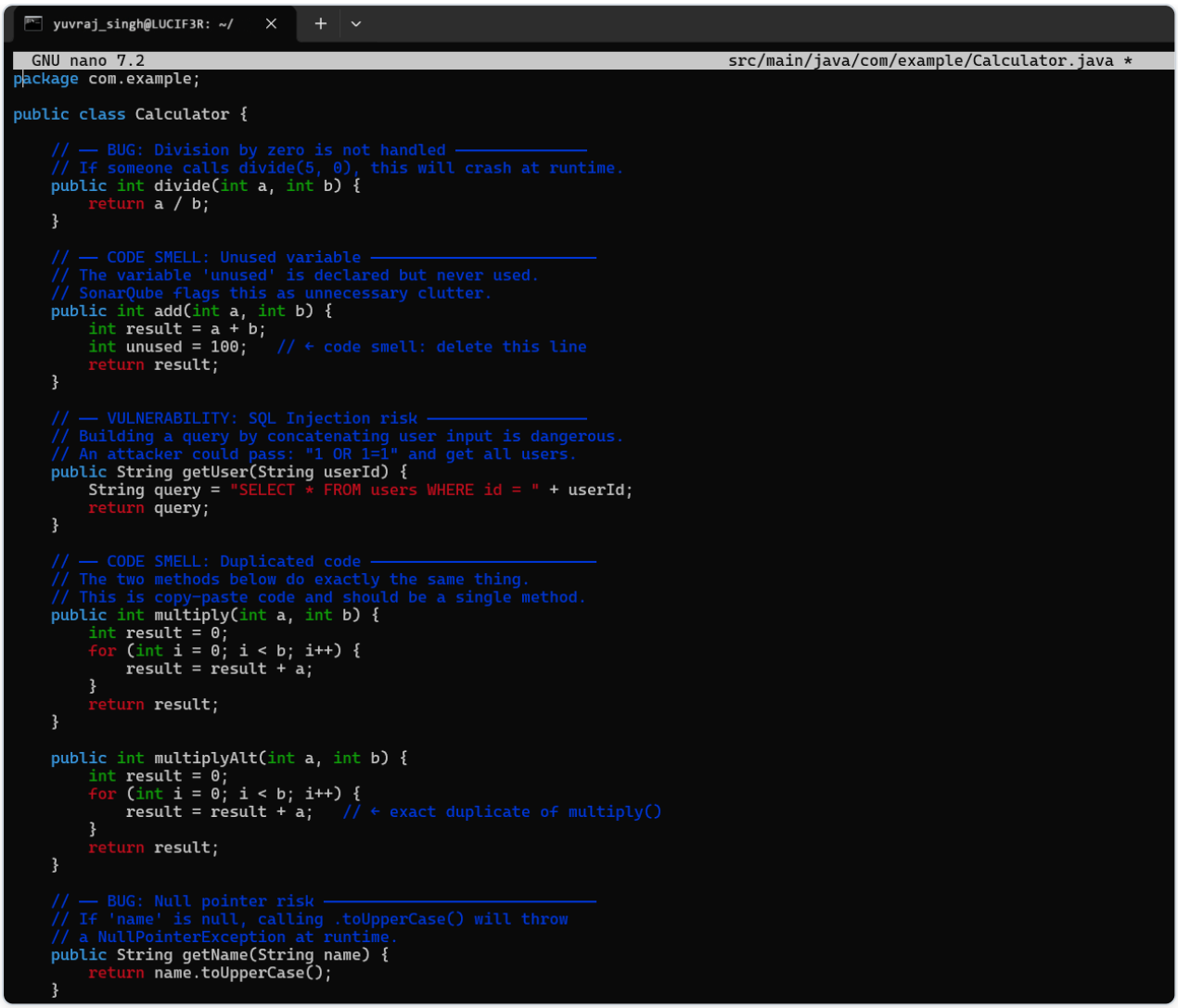
```
admin / admin
```

Step 2 — Create Sample Java App

Create project structure:

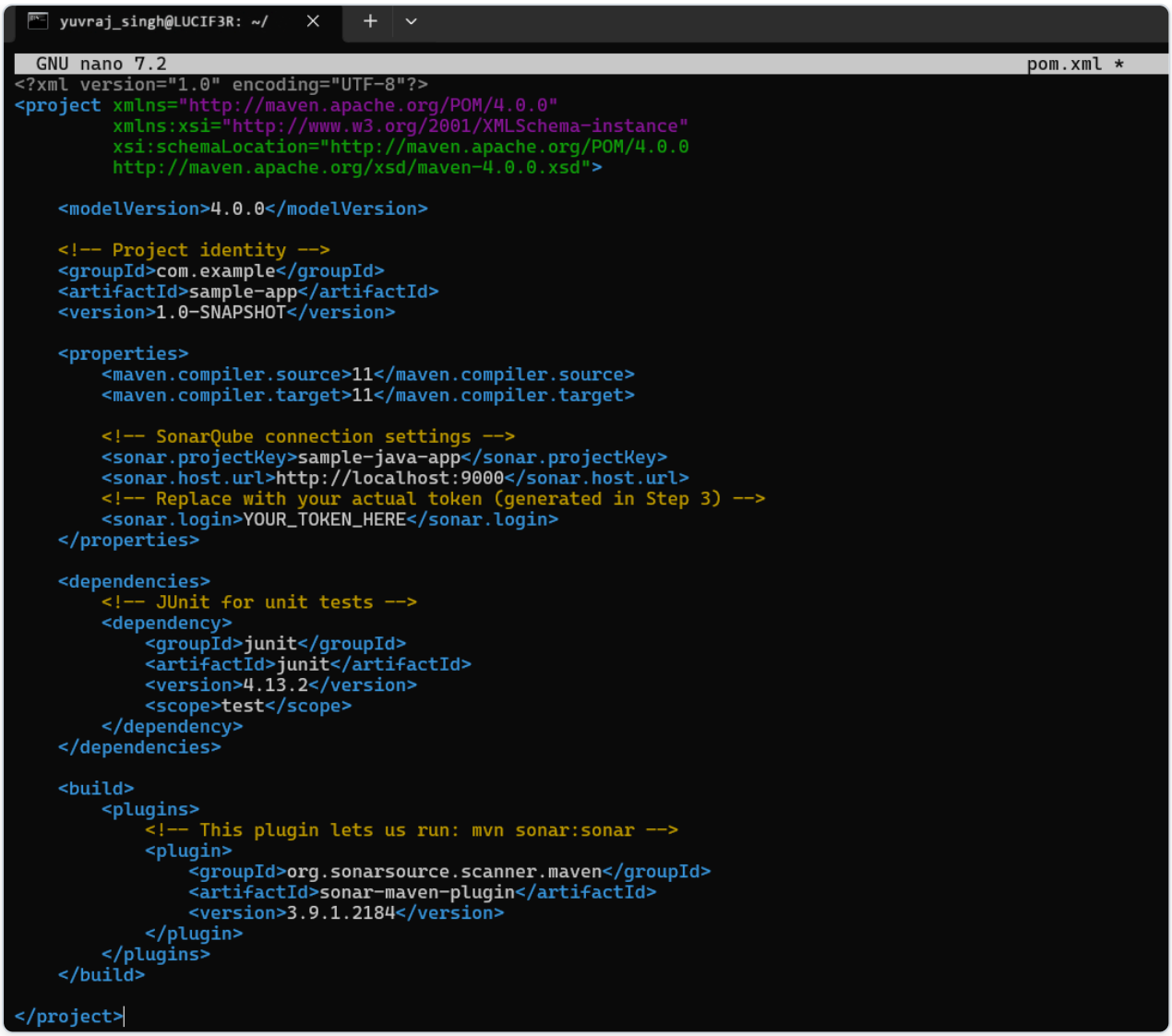


Create `Calculator.java`



Step 3 — Create Maven Config

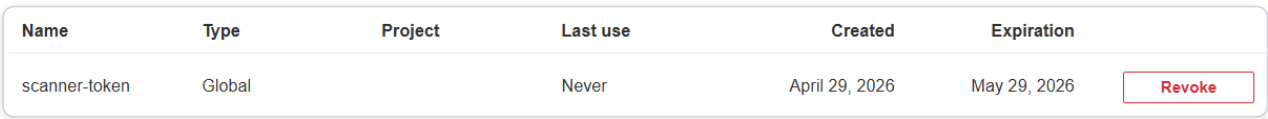
Create `pom.xml`



Step 4 — Generate Token

Open dashboard → profile → **My Account** → **Security**

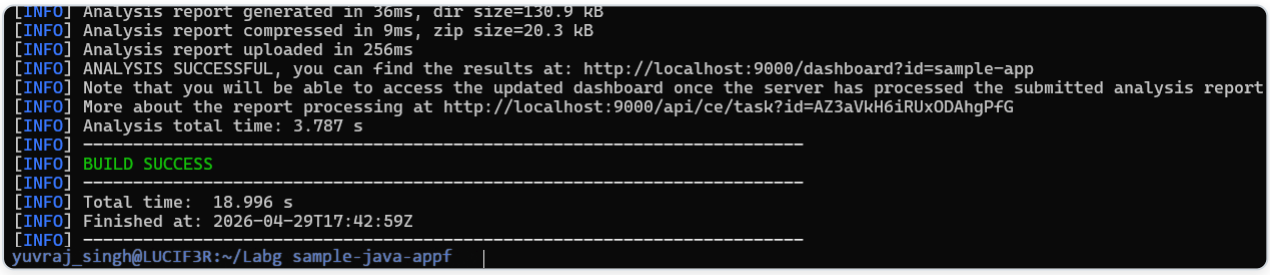
Generate token:



Step 5 — Run Scan

```
cd sample-java-app
```

Run scanner:

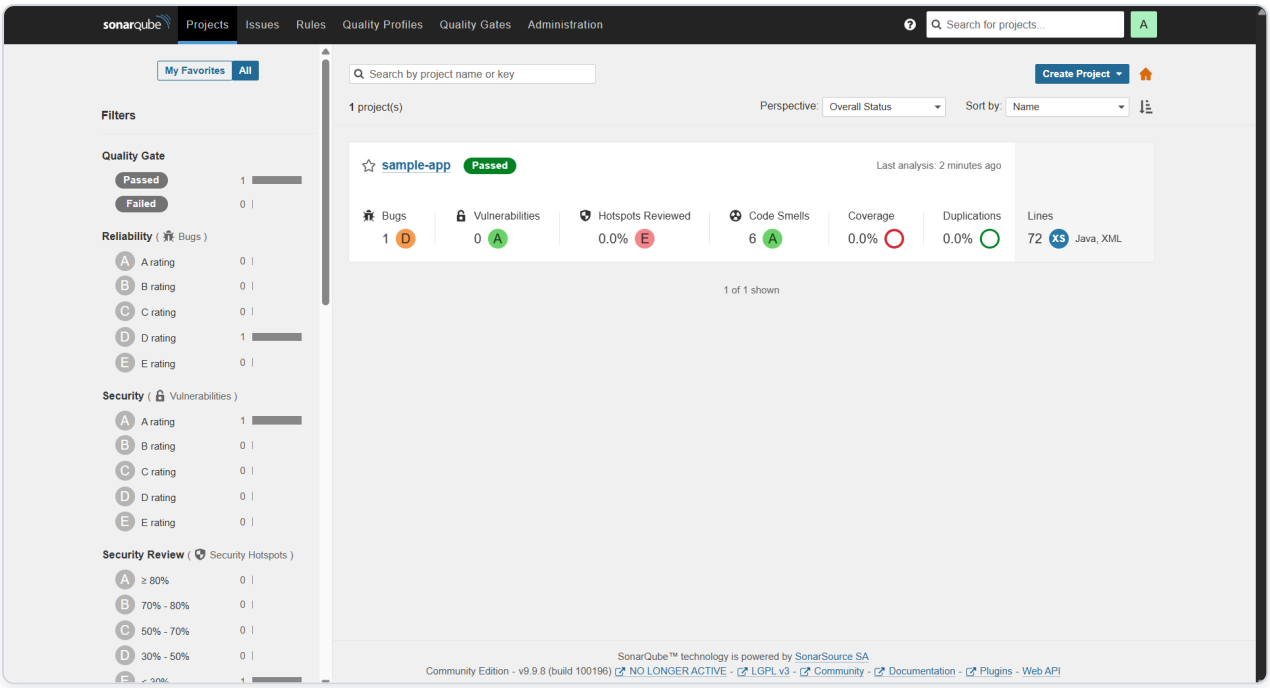


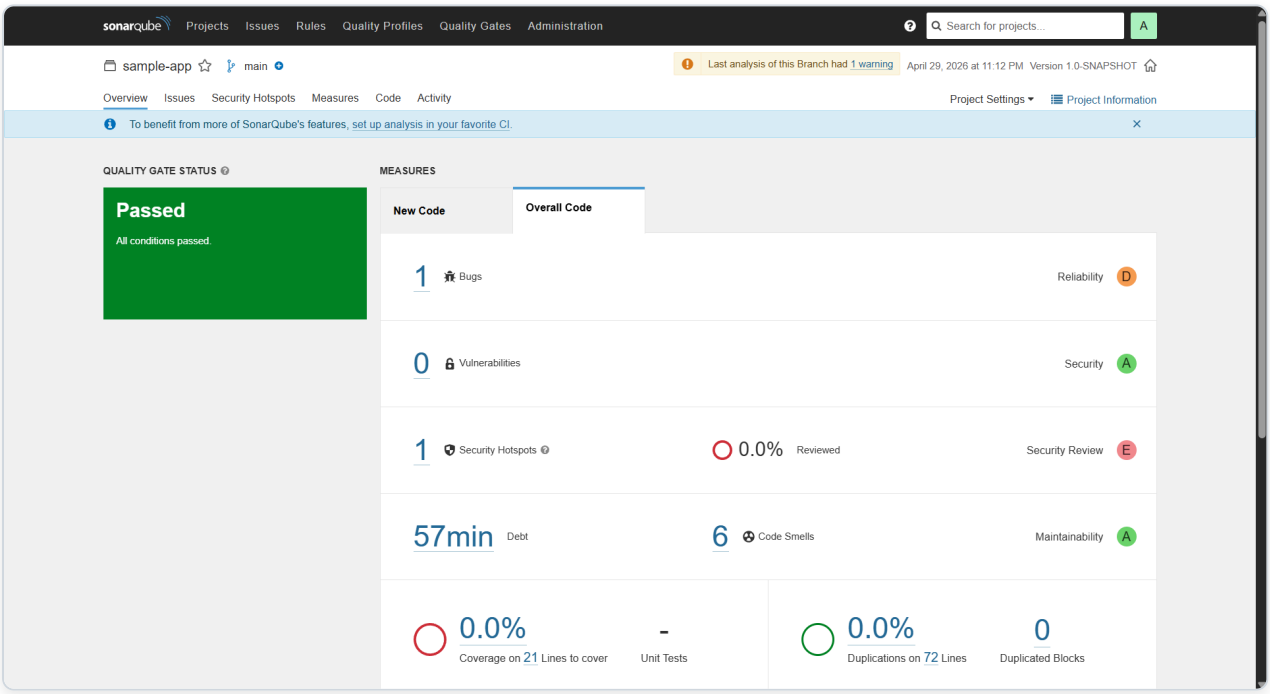
Process:

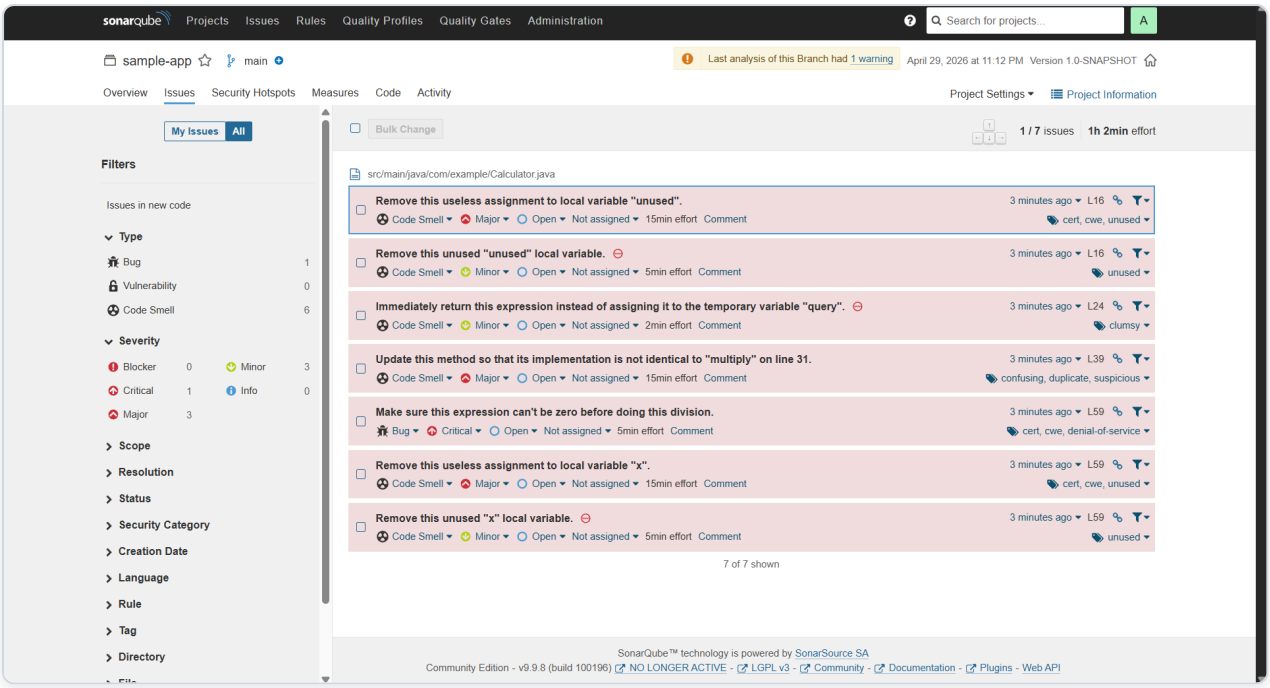
1. Maven compiles project
2. Scanner collects code
3. Sends analysis to server
4. Server evaluates rules
5. Results stored in PostgreSQL

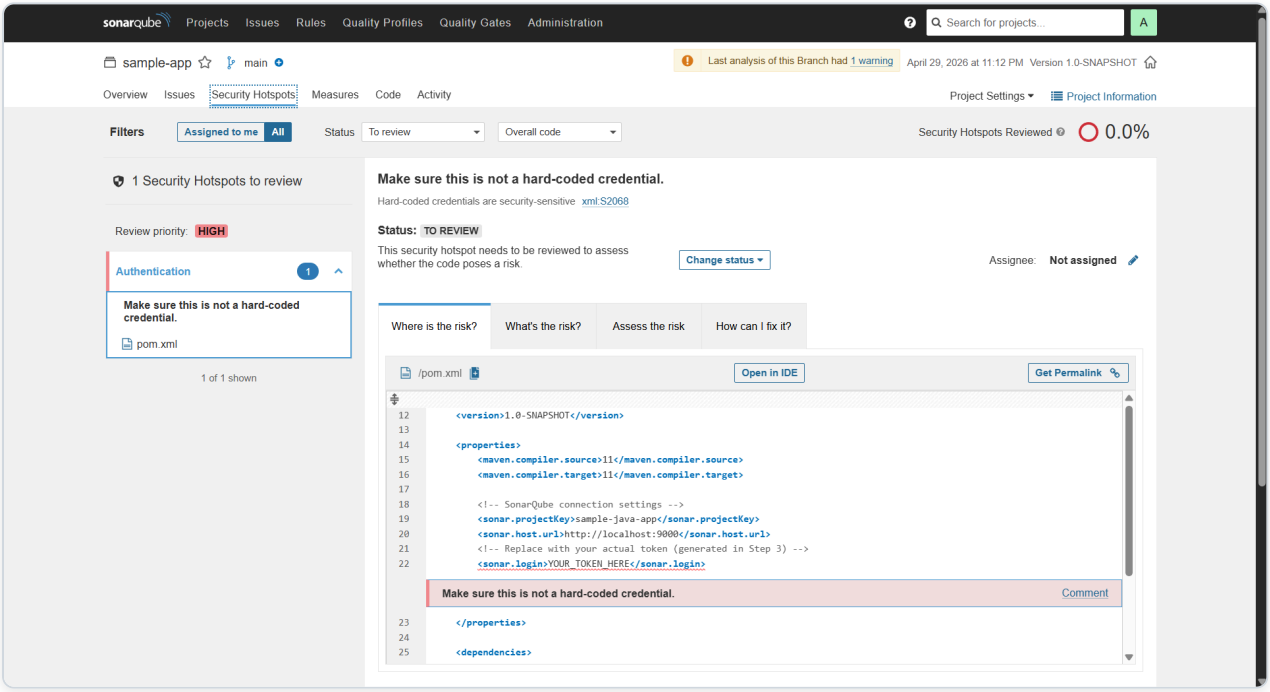
Step 6 — View Results

Open:


sonarqube
Projects
Issues
Rules
Quality Profiles
Quality Gates
Administration
Search for projects...
A
My Favorites
All
Search by project name or key
Create Project
Filters
1 project(s)
Perspective:
Overall Status
Sort by:
Name
Quality Gate
Passed
1
Failed
0
Reliability (Bugs)
A rating
0
B rating
0
C rating
0
D rating
1
E rating
0
Security (Vulnerabilities)
A rating
1
B rating
0
C rating
0
D rating
0
E rating
0
Security Review (Security Hotspots)
≥ 80%
0
70% - 80%
0
50% - 70%
0
30% - 50%
0
sample-app
Passed
Last analysis: 2 minutes ago
Bugs
1
D
Vulnerabilities
0
A
Hotspots Reviewed
0.0%
E
Code Smells
6
A
Coverage
0.0%
Duplications
0.0%
Lines
72
XS
Java, XML
1 of 1 shown
SonarQube™ technology is powered by SonarSource SA
Community Edition - v9.9.8 (build 100196) NO LONGER ACTIVE - LGPL v3 - Community - Documentation - Plugins - Web API


sonarqube
Projects
Issues
Rules
Quality Profiles
Quality Gates
Administration
Search for projects...
A
sample-app
main
Last analysis of this Branch had 1 warning
April 29, 2026 at 11:12 PM
Version 1.0-SNAPSHOT
Overview
Issues
Security Hotspots
Measures
Code
Activity
Project Settings
Project Information
To benefit from more of SonarQube's features, set up analysis in your favorite CI.
QUALITY GATE STATUS
Passed
All conditions passed.
MEASURES
New Code
Overall Code
1
Bugs
Reliability
D
0
Vulnerabilities
Security
A
1
Security Hotspots
0.0%
Reviewed
Security Review
E
57min
Debt
6
Code Smells
Maintainability
A
0.0%
Coverage on 21 Lines to cover
-
Unit Tests
0.0%
Duplications on 72 Lines
0
Duplicated Blocks





You will see:

- Bugs
- Vulnerabilities
- Code Smells
- Duplication
- Technical Debt
- Quality Gate

Results

Detected issues include:

- Division by zero bug
- SQL injection vulnerability
- Unused variable
- Duplicate methods
- Null pointer risk
- Empty catch block

Conclusion

In this experiment, SonarQube was deployed using Docker with PostgreSQL.
A Java project was analyzed using the Maven Sonar Scanner plugin, and issues such as bugs, vulnerabilities, and code smells were detected and visualized through the SonarQube dashboard.